🚀 Ultimate DSA Patterns Guide - Master Coding Interviews

A comprehensive collection of problem-solving patterns with LeetCode questions to accelerate your coding interview preparation.

1️⃣ Two Pointers Technique

Core Concept: Use two pointers moving through data to solve problems in O(n) time instead of O(n²).

Problems:

- Two Sum II - Input Array Is Sorted - Easy
- Remove Duplicates from Sorted Array - Easy
- Remove Element - Easy
- Squares of a Sorted Array - Easy
- 3Sum - Medium
- 3Sum Closest - Medium
- 3Sum Smaller - Medium 🔒
- Subarray Product Less Than K - Medium
- Sort Colors - Medium
- 4Sum - Medium
- Backspace String Compare - Easy
- Shortest Unsorted Continuous Subarray - Medium
- Container With Most Water - Medium

2️⃣ Fast & Slow Pointers (Floyd's Algorithm)

Core Concept: Use two pointers moving at different speeds to detect cycles or find middle elements.

Problems:

- Linked List Cycle - Easy
- Linked List Cycle II - Medium
- Happy Number - Easy
- Middle of the Linked List - Easy
- Palindrome Linked List - Easy

- Reorder List - Medium
- Circular Array Loop - Medium
- Find the Duplicate Number - Medium

3️⃣ Sliding Window Pattern

Core Concept: Maintain a window and slide it across the array/string to find optimal subarrays.

Problems:

- Maximum Average Subarray I - Easy
- Minimum Size Subarray Sum - Medium
- Longest Substring with At Most K Distinct Characters - Medium 🔒
- Fruit Into Baskets - Medium
- Longest Substring Without Repeating Characters - Medium
- Longest Repeating Character Replacement - Medium
- Max Consecutive Ones III - Medium
- Permutation in String - Medium
- Find All Anagrams in a String - Medium
- Minimum Window Substring - Hard
- Substring with Concatenation of All Words - Hard

4️⃣ Merge Intervals

Core Concept: Sort intervals and merge overlapping ones to solve scheduling/range problems.

Problems:

- Merge Intervals - Medium
- Insert Interval - Medium
- Interval List Intersections - Medium
- Non-overlapping Intervals - Medium
- Meeting Rooms - Easy 🔒
- Meeting Rooms II - Medium 🔒
- Maximum CPU Load - Hard 🔒
- Employee Free Time - Hard 🔒
- Minimum Number of Arrows to Burst Balloons - Medium

5️⃣ Cyclic Sort

Core Concept: Sort arrays containing numbers in a given range by placing each number at its correct index.

Problems:

- Missing Number - Easy
- Find All Numbers Disappeared in an Array - Easy
- Find the Duplicate Number - Medium
- Find All Duplicates in an Array - Medium
- Set Mismatch - Easy
- First Missing Positive - Hard
- Kth Missing Positive Number - Easy

6️⃣ In-place Linked List Reversal

Core Concept: Reverse LinkedList nodes without using extra space.

Problems:

- Reverse Linked List - Easy
- Reverse Linked List II - Medium
- Reverse Nodes in k-Group - Hard
- Swap Nodes in Pairs - Medium
- Rotate List - Medium
- Palindrome Linked List - Easy
- Reorder List - Medium

7️⃣ Stack Fundamentals

Core Concept: LIFO structure for tracking previous states and nested structures.

Problems:

- Valid Parentheses - Easy
- Min Stack - Medium
- Evaluate Reverse Polish Notation - Medium
- Generate Parentheses - Medium
- Next Greater Element I - Easy
- Next Greater Element II - Medium
- Simplify Path - Medium

- Basic Calculator - Hard
- Trapping Rain Water - Hard

8️⃣ Monotonic Stack

Core Concept: Maintain stack in increasing/decreasing order to find next greater/smaller elements.

Problems:

- Next Greater Element I - Easy
- Next Greater Element II - Medium
- Daily Temperatures - Medium
- Remove Nodes From Linked List - Medium
- Remove All Adjacent Duplicates In String - Easy
- Remove All Adjacent Duplicates in String II - Medium
- Remove K Digits - Medium
- Largest Rectangle in Histogram - Hard
- Maximal Rectangle - Hard

9️⃣ HashMap & HashSet

Core Concept: Use hash tables for O(1) lookups, frequency counting, and pattern matching.

Problems:

- Two Sum - Easy
- First Unique Character in a String - Easy
- Largest Unique Number - Easy 🔒
- Maximum Number of Balloons - Easy
- Longest Palindrome - Easy
- Ransom Note - Easy
- Group Anagrams - Medium
- Valid Sudoku - Medium
- Isomorphic Strings - Easy

🔟 Tree BFS (Level Order Traversal)

Core Concept: Use queue to traverse tree level by level.

Problems:

- Binary Tree Level Order Traversal - Medium
- Binary Tree Level Order Traversal II - Medium
- Binary Tree Zigzag Level Order Traversal - Medium
- Average of Levels in Binary Tree - Easy
- Minimum Depth of Binary Tree - Easy
- Maximum Depth of Binary Tree - Easy
- Populating Next Right Pointers in Each Node - Medium
- Binary Tree Right Side View - Medium
- Symmetric Tree - Easy

1️⃣1️⃣ Tree DFS (Depth First Search)

Core Concept: Use recursion/stack to explore tree paths from root to leaves.

Problems:

- Path Sum - Easy
- Path Sum II - Medium
- Sum Root to Leaf Numbers - Medium
- Path Sum III - Medium
- Diameter of Binary Tree - Easy
- Binary Tree Maximum Path Sum - Hard
- Lowest Common Ancestor of a Binary Tree - Medium
- Validate Binary Search Tree - Medium

1️⃣2️⃣ Graph Traversal (BFS & DFS)

Core Concept: Explore graphs using BFS (shortest path) or DFS (connectivity).

Problems:

- Find if Path Exists in Graph - Easy
- Number of Provinces - Medium
- Clone Graph - Medium
- Course Schedule - Medium
- Course Schedule II - Medium
- Pacific Atlantic Water Flow - Medium
- Number of Connected Components in an Undirected Graph - Medium 🔒
- Graph Valid Tree - Medium 🔒

1️⃣3️⃣ Matrix Traversal (Island Pattern)

Core Concept: Use DFS/BFS to explore connected components in 2D grids.

Problems:

- Number of Islands - Medium
- Max Area of Island - Medium
- Flood Fill - Easy
- Number of Closed Islands - Medium
- Island Perimeter - Easy
- Number of Distinct Islands - Medium 🔒
- Surrounded Regions - Medium
- Rotting Oranges - Medium

1️⃣4️⃣ Two Heaps Pattern

Core Concept: Use max-heap and min-heap together to find medians or balance datasets.

Problems:

- Find Median from Data Stream - Hard
- Sliding Window Median - Hard
- IPO - Hard
- Find Right Interval - Medium

1️⃣5️⃣ Subsets & Backtracking

Core Concept: Generate all possible combinations using recursive exploration.

Problems:

- Subsets - Medium
- Subsets II - Medium
- Permutations - Medium
- Permutations II - Medium
- Letter Case Permutation - Medium
- Generate Parentheses - Medium
- Generalized Abbreviation - Medium 🔒
- Combinations - Medium

1️⃣6️⃣ Modified Binary Search

Core Concept: Apply binary search on sorted/rotated arrays or search spaces.

Problems:

- Binary Search - Easy
- Search in Rotated Sorted Array - Medium
- Find Minimum in Rotated Sorted Array - Medium
- Search a 2D Matrix - Medium
- Find Peak Element - Medium
- Find First and Last Position of Element in Sorted Array - Medium
- Minimum Number of Days to Make m Bouquets - Medium
- Koko Eating Bananas - Medium
- Capacity To Ship Packages Within D Days - Medium
- Median of Two Sorted Arrays - Hard

1️⃣7️⃣ Bitwise XOR

Core Concept: Use XOR properties (a^a=0, a^0=a) to find unique elements.

Problems:

- Single Number - Easy
- Single Number II - Medium
- Single Number III - Medium
- Complement of Base 10 Integer - Easy
- Flipping an Image - Easy
- Number of 1 Bits - Easy

1️⃣8️⃣ Top K Elements (Heap/Priority Queue)

Core Concept: Use heaps to efficiently find K largest/smallest elements.

Problems:

- Kth Largest Element in an Array - Medium
- K Closest Points to Origin - Medium
- Top K Frequent Elements - Medium
- Sort Characters By Frequency - Medium
- Kth Largest Element in a Stream - Easy
- Find K Closest Elements - Medium

- Reorganize String - Medium
- Task Scheduler - Medium
- Maximum Frequency Stack - Hard

1️⃣9️⃣ K-Way Merge

Core Concept: Merge K sorted arrays/lists using a min-heap.

Problems:

- Merge k Sorted Lists - Hard
- Kth Smallest Element in a Sorted Matrix - Medium
- Smallest Range Covering Elements from K Lists - Hard
- Find K Pairs with Smallest Sums - Medium

2️⃣0️⃣ Greedy Algorithms

Core Concept: Make locally optimal choices to find global optimum.

Problems:

- Valid Palindrome II - Easy
- Maximum Length of Pair Chain - Medium
- Minimum Add to Make Parentheses Valid - Medium
- Remove Duplicate Letters - Medium
- Partition Labels - Medium
- Jump Game - Medium
- Jump Game II - Medium
- Gas Station - Medium

2️⃣1️⃣ Dynamic Programming - 0/1 Knapsack

Core Concept: Choose to include/exclude items to maximize value within constraints.

Problems:

- Partition Equal Subset Sum - Medium
- Target Sum - Medium
- Last Stone Weight II - Medium
- Ones and Zeroes - Medium

- Coin Change - Medium
- Coin Change II - Medium

2️⃣2️⃣ Backtracking Deep Dive

Core Concept: Explore all possibilities with pruning for optimization problems.

Problems:

- Combination Sum - Medium
- Combination Sum II - Medium
- Combination Sum III - Medium
- Word Search - Medium
- Word Search II - Hard
- Sudoku Solver - Hard
- N-Queens - Hard
- Palindrome Partitioning - Medium

2️⃣3️⃣ Trie (Prefix Tree)

Core Concept: Tree structure for efficient prefix-based string operations.

Problems:

- Implement Trie (Prefix Tree) - Medium
- Design Add and Search Words Data Structure - Medium
- Word Search II - Hard
- Replace Words - Medium
- Search Suggestions System - Medium
- Longest Word in Dictionary - Medium

2️⃣4️⃣ Topological Sort

Core Concept: Linear ordering of vertices in directed acyclic graph (DAG).

Problems:

- Course Schedule - Medium
- Course Schedule II - Medium
- Alien Dictionary - Hard 🔒

- Sequence Reconstruction - Medium 🔒
- Minimum Height Trees - Medium
- Find All Possible Recipes from Given Supplies - Medium

2️⃣5️⃣ Union Find (Disjoint Set)

Core Concept: Efficiently track and merge disjoint sets to detect connectivity.

Problems:

- Redundant Connection - Medium
- Number of Provinces - Medium
- Graph Valid Tree - Medium 🔒
- Number of Islands II - Hard 🔒
- Accounts Merge - Medium
- Satisfiability of Equality Equations - Medium
- Smallest String With Swaps - Medium

2️⃣6️⃣ Ordered Set/Map

Core Concept: Maintain sorted order for efficient range queries and operations.

Problems:

- Contains Duplicate III - Hard
- My Calendar I - Medium
- My Calendar II - Medium
- My Calendar III - Hard
- Exam Room - Medium
- Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit - Medium

---

💡 Pro Tips for Success

1. **Master the Pattern First:** Understand the core concept before jumping to problems
2. **Start with Easy:** Build confidence with easier problems before tackling hard ones
3. **Practice Consistently:** Solve 2-3 problems daily from different patterns
4. **Time Yourself:** Practice with interview time constraints (45-60 mins per problem)
5. **Review Solutions:** Study optimal solutions even after solving correctly
6. **Space-Time Trade-offs:** Always consider both complexity metrics

7. **Edge Cases Matter:** Test with empty inputs, single elements, and large datasets
8. **Code Clean:** Write production-quality code with proper variable names